

Assignment 1 — Build a Custom Missing-Value Imputer

Feature Engineering & MLOps · Unit 1, Session 4 follow-up (Missing Values)

Course	Feature Engineering & MLOps
Assignment No.	1
Topic	Custom Imputer Class (Missing Value Handling)
Marks	____ / 100
Due Date	_____

1. Objective

Design and implement your own Python class that imputes (fills in) missing values in a dataset, following the same fit/transform pattern used by scikit-learn's built-in transformers (like `SimpleImputer`). You will then apply your class to the PrepEdge coaching-institute dataset from Unit 1 and verify it works correctly.

This assignment tests whether you understand **why** imputation works the way it does (Unit 1, Session 4) — not just how to call a library function. Building the class yourself forces you to make every decision explicit: what statistic to use, when to compute it, and how to avoid data leakage.

2. Learning Outcomes

- Understand and implement scikit-learn's estimator API (`fit` / `transform` pattern).
- Apply object-oriented programming (classes, constructors, stored/fitted attributes) to a real data-processing problem.
- Correctly distinguish numeric vs. categorical columns and impute each appropriately.
- Practice strict train/test discipline: fitted statistics must come from training data only.
- Extend basic imputation with a missing-value indicator feature, connecting back to the MCAR/MAR/MNAR discussion from Session 4.

3. Dataset

Use `data/raw/student_performance_raw.csv` from your Unit 1 repository (the same PrepEdge dataset used in Session 4 — Missing Values). It contains missing values under all three mechanisms you studied:

Column	Missingness type	Data type
<code>weekly_study_hours</code>	MCAR (random)	Numeric (continuous)

Column	Missingness type	Data type
prev_exam_score	MAR (linked to attendance_pct)	Numeric (continuous)
mock_test_3	MNAR (linked to its own low values)	Numeric (continuous)
income_bracket	Missing categorical values	Categorical (ordinal)
feedback_text	Missing free-text values	Text

NOTE: If you no longer have this file, regenerate it with `python data/generate_dataset.py` from your Unit 1 repo — it uses a fixed random seed, so everyone's data will be identical.

4. Tasks

4.1 — Design the class

Create a class called `CustomImputer` that inherits from `sklearn.base.BaseEstimator` and `sklearn.base.TransformerMixin`. Your class must satisfy every requirement below:

- **Constructor** — accept at least these parameters: `numeric_strategy` ('mean' or 'median', default 'median'), `categorical_strategy` (default 'most_frequent'), and `add_missing_indicator` (boolean, default True).
- `fit(X, y=None)` — for every column in `X`, compute and STORE the fill value that will later be used to replace missing entries (mean/median for numeric columns, the mode for categorical columns). Store these as a fitted attribute, e.g. `self.fill_values_`, following scikit-learn's convention of a trailing underscore for attributes learned during `fit`. Return `self`.
- `transform(X)` — return a COPY of `X` with missing values filled in using the values stored during `fit` (never recompute statistics inside `transform`). If `add_missing_indicator=True`, add one new binary column per column that had missing values in the training data, named `<column>_was_missing`.
- **Auto-detect column type** — your class should automatically decide whether a column is numeric or categorical (e.g. using `pandas.api.types`) rather than requiring the user to specify it manually.
- **Guard rail** — calling `transform()` before `fit()` must raise a clear, informative error (hint: check scikit-learn's `check_is_fitted` utility).
- **Docstrings** — document the class and every method (purpose, parameters, returns).

4.2 — Apply it to the dataset

1. Load `student_performance_raw.csv` and split it into train (80%) and test (20%) sets using `train_test_split`.
2. Fit your `CustomImputer` on the TRAINING split only.
3. Use the fitted imputer to transform BOTH the training and test splits.
4. Verify programmatically that no missing values remain in the imputed columns on either split (e.g. `assert .isna().sum().sum() == 0` on the relevant columns).
5. Report, in a small table, the mean and standard deviation of each numeric column BEFORE and AFTER imputation, and briefly comment on what changed.
6. **Sanity check:** run scikit-learn's own `SimpleImputer` with an equivalent strategy on the same numeric columns, and confirm your class produces the same fill values. This is your evidence that the core logic is implemented correctly.

4.3 — Reflection questions (short answer, 3–4 sentences each)

1. Why must `fit()` be called only on the training split, and never on the full dataset or the test split?
2. `mock_test_3` is MNAR (Session 4). Does mean/median imputation genuinely solve the problem for this column? What does your `add_missing_indicator` feature contribute that plain imputation does not?
3. Suppose a brand-new column, entirely missing in the training data but present in the test data, is passed to your imputer. What does your current implementation do — and what SHOULD a production-grade version do instead?

5. Submission Instructions

1. In your existing course GitHub repository (the one you set up in Unit 1), create a new top-level folder named exactly `Assignments` (if it does not already exist).
2. Inside `Assignments`, add your completed notebook as `Assignment1_CustomImputer.ipynb`. Make sure every cell has been run top-to-bottom with no errors, and outputs are visible (do not clear outputs before committing).
3. Include your class definition, the applied example, your verification checks, and written answers to the reflection questions — all inside this one notebook, clearly separated with markdown headers.
4. Commit and push your changes:

```
git add Assignments/Assignment1_CustomImputer.ipynb
git commit -m "Assignment 1: Custom Imputer class"
git push
```

Finally, share your GitHub repository link with the instructor by the due date above. Grading is done directly from the `Assignments` folder in your repo — make sure your notebook is there and pushed before the deadline.

IMPORTANT: Only the notebook committed to your `Assignments` folder on GitHub will be evaluated. Local-only or emailed notebooks will not be graded.

6. Grading Rubric

Criteria	Weight
Correct class structure & scikit-learn API compliance (BaseEstimator/TransformerMixin, fit returns self, transform returns a copy)	25%
Correct fit/transform logic, including strictly train-only fitting	25%
Successful application to the dataset with working verification checks	20%
Missing-indicator feature implemented and its MNAR relevance explained	10%
Reflection questions answered thoughtfully	10%
Code quality: docstrings, clear variable names, organized markdown	10%

7. Bonus (Optional, +10% extra credit)

Extend your `CustomImputer` to accept a per-column strategy override, e.g.:

```
CustomImputer(  
    numeric_strategy='median',  
    categorical_strategy='most_frequent',  
    column_overrides={  
        'weekly_study_hours': 'mean',  
        'income_bracket': 'most_frequent',  
    }  
)
```

so that individual columns can use a different strategy than the dataset-wide default. Demonstrate it working on at least two columns with different strategies.

Feature Engineering & MLOps · Assignment 1 · Refer to Unit 1, Session 4 (Missing Values) notebook for the underlying concepts if you get stuck.